TU850
Algorithms and Data Structures

Assignment 1

Project Report

Lea Stanisavljevic
A00046044

School of Computer Science
Technological University Dublin

21/03/2026

Declaration

I hereby declare that the work described in this report is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Stanisavljevic

Lea Stanisavljevic

21/03/2026

Table of Contents

1. **INTRODUCTION** 4
2. **DATASET DESCRIPTION** 4
3. **METHODOLOGY** 4
4. **EXPERIMENTAL DESIGN** 5
5. **RESULTS** 6
6. **CONCLUSION** 7
APPENDICES 8
REFERENCES 14

1. Introduction

This assignment's main goal is to implement and compare five sorting algorithms: bubble sort, insertion sort, selection sort, quick sort and merge sort. The goal is to find out how long each algorithm takes to sort the data from the csv file and to compare the real results with their expected time complexities. This helps us see how algorithms perform in real situations and recognise the difference between less efficient algorithms like ($O(n^2)$) and more efficient ones like ($O(n \log n)$).

Sorting algorithms are techniques that help organise data in a certain order, like for example from smallest to largest or largest to smallest. A list of numbers such as [2, 3, 5, 1, 4] can be arranged to become [1, 2, 3, 4, 5]. These algorithms are commonly used in computer science because many systems need sorted data to function well. In real world systems like databases or online platforms we need to process large amounts of data quickly. If the sorting algorithm is not efficient, the system may slow down, especially when the dataset increases in size. Analysing how efficient algorithms are is important because it helps us figure out which algorithms work well with small datasets and which ones work better for bigger ones and then we can use what fits best.

2. Dataset Description

The dataset that was used in this assignment represents a used car inventory and was stored as a csv file. The rows in the dataset represent a vehicle and include several attributes like id, brand, model, year it was manufactured, mileage, engine size and price. I used the PyChram IDE for this assignment. The dataset was loaded using csv.DictReader function and each row was converted into a Vehicle object. This made it easier to manage and work with the data in an organised way. Before sorting the data was preprocessed by converting values into the correct types: year of manufacture into an integer and mileage, engine size, and price into floats inside the Vehicle class constructor. This ensured that sorting operations were performed correctly using numerical comparisons rather than string-based comparisons. The main attributes used for sorting were price (used to test how algorithms perform as dataset size increases), mileage (used when filtering vehicles by year) and year (used to group the data for the second experiment).

3. Methodology

Bubble Sort ($O(n^2)$)
Compares two neighbouring elements and swaps them if they are not in the correct order. This step keeps happening until the entire list is sorted. Since it checks each element over and over, it gets really slow when dealing with large datasets like this one. Bubble sort took 571 seconds for dataset size 64000 which is nearly 10 minutes runtime.

Insertion Sort ($O(n^2)$)
Creates a sorted list by adding one element at a time and putting each element in the right place. It works better than bubble sort for small or almost sorted data, but it doesn't do well with larger datasets as performance drops and runtime gets longer.

Selection Sort ($O(n^2)$)
Repeatedly looks for the smallest item in the part of the list that is not sorted and places it in the right spot. It does a fixed number of comparisons, so it’s not very efficient for big datasets.

Quick Sort ($O(n \log n)$)
An algorithm that uses a divide and conquer approach. It picks a pivot and divides the dataset into smaller and larger values, then sorts each part again using the same method. It is much quicker than quadratic algorithms and works well with large datasets like this one.

Merge Sort ($O(n \log n)$)
Also uses the divide and conquer approach. The process divides the dataset into smaller sections, organises them and then combines them again. It performs consistently and is dependable no matter the order of the input.

4. Experimental Design

Dataset Size Scaling
The dataset was tested with increasing sizes: 250, 500, 1000, 2000, 8000, 16000, 32000, 64000. This helped see how each algorithm performed as the dataset got bigger.

Timing Method
The time taken to execute was measured using time.perf_counter() which gives precise runtime to measure how well the code performs.

Repetition and Averaging
Each sorting algorithm was run three times for each dataset size and then calculated the average runtime. This helps reduce random variation in results and makes them more accurate/reliable.

Fair Testing Conditions
To make sure the comparison is fair all algorithms used the same dataset, it was mixed up before sorting to prevent any bias, every algorithm operated on a duplicate of the data.

Arranging by Year
A second experiment involved filtering the dataset for the years 2020 to 2024 and then sorting the results based on mileage. This helped see how well algorithms work on smaller parts of the data.

5. Results

Sorting by Price (Increasing Dataset Size)

Number of vehicles	Bubble Sort – Average Runtime (seconds)	Selection Sort – Average Runtime (seconds)	Insertion Sort – Average Runtime (seconds)	Quick Sort – Average Runtime (seconds)	Merge Sort – Average Runtime (seconds)
1 to 250	0.005740390000000002	0.0024619673333333334	0.0023572940000000284	0.0007027769999999878	0.0010194700000000223
1 to 500	0.023344501	0.010804463333333366	0.010411993000000027	0.0014432109999999998	0.0020299776666666824
1 to 1000	0.12087670300000004	0.04586616866666667	0.04648171099999998	0.00289813733333365	0.0044287903333332474
1 to 2000	0.4307168159999999	0.1868558513333333	0.1937239716666664	0.006231113000000417	0.01007551633333034
1 to 8000	7.9857529513333345	3.500713459666668	3.3290570849999974	0.032744663666666916	0.05940677966666641
1 to 16000	32.248093669	12.342726782333338	12.652396544333328	0.06349120666665915	0.10804478233333725
1 to 32000	130.12307579199998	57.998056507	62.13997258833331	0.12984624933331665	0.1958837689999958
1 to 64000	571.2579246403333	279.7189957279999	273.31582049466687	0.2986921496667492	0.48256097233388573

The results show a noticeable difference in the way the algorithms perform as the size of the dataset grows. Bubble sort, selection sort and insertion sort all have a sharp increase in runtime, especially when the number of elements exceeds 8000. For example, bubble sort takes about 8 seconds with 8000 elements and jumps to over 570 seconds with 64000 elements, showing that it does not scale well. On the other hand, quick sort and merge sort have slower and more consistent increases in runtime. Even with the biggest dataset, quick sort stays below 0.3 seconds and merge sort is under 0.5 seconds, showing how efficient they are even as the dataset grows. In general, quadratic algorithms ($O(n^2)$) are not practical and efficient for large datasets. Quick sort and merge sort ($O(n \log n)$) are efficient and can handle larger data very well.

Sorting by Mileage (Filtered by Year)

Number of vehicles	Bubble Sort – Average Runtime (seconds)	Selection Sort – Average Runtime (seconds)	Insertion Sort – Average Runtime (seconds)	Quick Sort – Average Runtime (seconds)	Merge Sort – Average Runtime (seconds)
2020	1.7243623783330502	0.7430386519999956	0.7962038299998918	0.030603840000064036	0.03409475566695619
2021	1.776084056666453	0.7272900220001853	0.7627771973335863	0.013006116000118103	0.020268180667092867
2022	1.7905223940000117	0.7722665479999099	0.8346550650003337	0.013826217333189561	0.021716339333579526
2023	1.6503524630000659	1.2225908483333114	0.8653466626665249	0.022311766999943455	0.07690170033341322

2024	1.8258793063332632	0.7785710146663405	0.9594000906669559	0.018183014333468844	0.02744383166676319

The results clearly show that quick sort and merge sort are the fastest algorithms, showing very low runtimes when compared to the other algorithms. On the other hand bubble sort, selection sort and insertion sort are much slower, even though the dataset sizes for each year are quite small. The differences between the algorithms are not as big as in the large dataset experiment, but the general trend is still somewhat unchanged. Quadratic algorithms, which have a time complexity of $O(n^2)$, take significantly longer to run. In contrast, quick sort and merge sort, with a time complexity of O(n log n) are much more efficient. The results show that more efficient algorithms perform better, regardless of how the data is organised, even when using smaller chunks of the dataset.

6. Conclusion

The results show how different sorting algorithms perform with actual data. The experiments showed that algorithms with a time complexity of $O(n^2)$, like bubble sort, selection sort and insertion sort, get really slow runtime when the dataset size gets bigger. In the first experiment, bubble sort took more than 570 seconds to sort 64,000 elements. This shows that bubble sort does not work well with large datasets. In contrast, quick sort and merge sort showed significantly better performance. Their time complexity of O(n log n) helps them work well with large datasets, which is why their runtime stays low even with the biggest dataset sizes. For example quick sort took less than 0.3 seconds, while merge sort took under 0.5 seconds. This shows a significant difference when compared to the quadratic algorithms. The second experiment involved filtering the data by year and sorting it by mileage and it showed similar patterns. Although the datasets were smaller, quick sort and merge sort remained the fastest, while the other algorithms were clearly slower. The efficiency of an algorithm is important no matter how big the dataset is but the difference is clearer when the dataset gets larger. The experimental results were in line with what we expected based on theory. Understanding time complexity is important when selecting algorithms, especially in real world systems where performance is even more important. Basic algorithms can be effective for small datasets but when dealing with larger datasets, it's obvious that more efficient algorithms are the preferred option.

All in all this assignment really helped me apply what we learned in lectures and tutorials. It improved my understanding of the concepts we covered in class through hands-on experience.

Limitation

A limitation of this experiment was that very large dataset sizes led to very long runtimes for quadratic algorithms, making it impractical to test them repeatedly. Only randomly shuffled data was used, so we did not look at how sorted or reverse sorted data would affect the results. The results were still consistent and clearly showed the differences in how the algorithms performed.

Appendices

```
Dataset size: 250
Bubble: 0.006893650000000011
Selection: 0.0024619673333333334
Insertion: 0.0026660966666666544
Quick: 0.00083850433333336
Merge: 0.0010194700000000223

Dataset size: 500
Bubble: 0.022626967666666647
Selection: 0.010804463333333366
Insertion: 0.010411993000000027
Quick: 0.0014432109999999998
Merge: 0.0020299776666666824

Dataset size: 1000
Bubble: 0.10492397399999998
Selection: 0.044738061333333246
Insertion: 0.04536517666666673
Quick: 0.003098899000000044
Merge: 0.0043439929999999904

Dataset size: 2000
Bubble: 0.4307168159999999
Selection: 0.18380390866666652
Insertion: 0.18877293866666664
Quick: 0.006353780666666846
Merge: 0.010132791333333557
```

```
Dataset size: 8000
Bubble: 7.914435979
Selection: 2.994272705666667
Insertion: 3.374157364666665
Quick: 0.029004846999998318
Merge: 0.047003414333332216

Dataset size: 16000
Bubble: 32.248093669
Selection: 12.342726782333338
Insertion: 12.652396544333328
Quick: 0.06395560099999405
Merge: 0.10416751166666434

Dataset size: 32000
Bubble: 144.0818595246667
Selection: 69.3636876506667
Insertion: 62.13997258833331
Quick: 0.13728837599993918
Merge: 0.1958837689999958

Dataset size: 64000
Bubble: 571.2579246403333
Selection: 279.7189957279999
Insertion: 276.30653976799994
Quick: 0.2986921496667492
Merge: 0.47365566366655304
```

```
Year: 2020
Bubble: 1.7753337129997817
Selection: 0.7610947990006025
Insertion: 0.8342834929999299
Quick: 0.015593903666740516
Merge: 0.025469979000263265

Year: 2021
Bubble: 1.7000358943335716
Selection: 0.7137137116663629
Insertion: 0.8380904230001155
Quick: 0.013621450666808718
Merge: 0.030239901666391233

Year: 2022
Bubble: 1.7905223940000117
Selection: 0.7825358273336557
Insertion: 0.8522219833333414
Quick: 0.013701906666938157
Merge: 0.021791401666632737

Year: 2023
Bubble: 1.7626340583328783
Selection: 0.6829593363336244
Insertion: 0.7242668040001566
Quick: 0.013630237000143097
Merge: 0.021619131000079506

Year: 2024
Bubble: 1.8258793063332632
Selection: 0.7430793570001697
Insertion: 0.9523841779997989
Quick: 0.01542926966703817
Merge: 0.02818794166675313
```

sorting.py

```
def bubble_sort(arr): # repeatedly compares adjacent elements and
swaps them if needed (O(n²))
    arr = arr.copy() # creates a copy of the data to avoid modifying
the original data
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:# stops early if the data is already sorted
            break
    return arr

def insertion_sort(arr): # insertion sort algorithm (O(n²))
    # sorted list one element at a time
    arr = arr.copy()
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key # insert key into correct position
    return arr

def selection_sort(arr): # selection sort algorithm (O(n²))
    # repeatedly selects the smallest element and places it in
correct position
    arr = arr.copy()
    n = len(arr)
    for i in range(n):
        min_index = i # assumes the current index is minimum
        for j in range(i + 1, n): # finds actual minimum in remaining
list
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i] # swaps with
the min value found
    return arr

def quick_sort(arr): # quick sort algorithm (O(n log n))
    # uses divide and conquer approach
    arr = arr.copy()
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    # splits into three parts
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right) # sorts and
combines

def merge_sort(arr): # merge sort algorithm (O(n log n))
    # splits and merges lists
    arr = arr.copy()
    if len(arr) <= 1:
        return arr
```

```
    mid = len(arr) // 2 # splits list into halves
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right) # merge the sorted halves

def merge(left, right): # helper function for merge sort
    merged = []
    i = j = 0
    while i < len(left) and j < len(right): # merges elements in
sorted order
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    # adds the remaining elements
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

vehicle.py

```
class Vehicle: #single vehicle in the dataset
    def __init__(self, vehicle_id, brand, model, year, mileage,
engine_size, price):
        self.vehicle_id = vehicle_id
        self.brand = brand
        self.model = model
        self.year = int(year)
        self.mileage = float(mileage)
        self.engine_size = float(engine_size)
        self.price = float(price)
# stores all the attributes and then converts year to integer and the
other values to floats

    def __str__(self): # string representation of the object for when
its printing
        return f"{self.brand} {self.model} ({self.year}) -
€{self.price}"

# stores and manages the vehicles in one collection
class VehicleCollection:
    def __init__(self): # list to store all the vehicle objects
        self.vehicles = []

    def add_vehicle(self, vehicle): # adds an object into the
collection
        self.vehicles.append(vehicle)

    def get_attribute_list(self, attribute): # returns value for
specific attribute
        return [getattr(v, attribute) for v in self.vehicles]

    def filter_by_year(self, year): # filters by year
        return [v for v in self.vehicles if v.year == year]
```

experiment.py

```
import time

# function to benchmark a sorting algorithm
def benchmark_sort(sort_function, data, repetitions=3):
    total_time = 0 # runs the algorithm multiple times and returns
average runtime

    for _ in range(repetitions):
        test_data = data.copy()
        start = time.perf_counter() # starts the timer
        sort_function(test_data) # runs the sorting algorithms
        end = time.perf_counter()  # ends the timer
        total_time += (end - start) # adds the elapsed time
    return total_time / repetitions # returns the average runtime
```

main.py

```
# imports libraries needed for the project to run
import csv
import random
from vehicle import Vehicle, VehicleCollection
from sorting import *
from experiment import benchmark_sort

def load_data(filename): # loads the dataset from the csv file
    collection = VehicleCollection()

    with open(filename) as file:
        reader = csv.DictReader(file)

        print(reader.fieldnames)

        for row in reader: # loops through each row to create objects
            vehicle = Vehicle(
                row["Vehicle_ID"],
                row["Brand"],
                row["Model"],
                row["Year"],
                row["Mileage"],
                row["Engine_Size"],
                row["Price"]
            )
            collection.add_vehicle(vehicle)
    return collection


# experiment 1: tests sorting performance based on price with
increasing dataset sizes
def experiment_by_price(collection):
    sizes = [250, 500, 1000, 2000, 8000, 16000, 32000, 64000]
    prices = collection.get_attribute_list("price")

    for size in sizes: # extracts all price values
        subset = prices[:size]
        random.shuffle(subset) # shuffles to avoid bias

        print(f"\nDataset size: {size}")
        print("Bubble:", benchmark_sort(bubble_sort, subset))
        print("Selection:", benchmark_sort(selection_sort, subset))
        print("Insertion:", benchmark_sort(insertion_sort, subset))
```

```python
        print("Quick:", benchmark_sort(quick_sort, subset))
        print("Merge:", benchmark_sort(merge_sort, subset))

# experiment 2: test sorting by mileage after filtering by year
def experiment_by_year(collection):
    years = [2020, 2021, 2022, 2023, 2024]

    for year in years:
        filtered = collection.filter_by_year(year)
        mileage = [v.mileage for v in filtered]

        print(f"\nYear: {year}") # benchmarks each sorting algorithm
        print("Bubble:", benchmark_sort(bubble_sort, mileage))
        print("Selection:", benchmark_sort(selection_sort, mileage))
        print("Insertion:", benchmark_sort(insertion_sort, mileage))
        print("Quick:", benchmark_sort(quick_sort, mileage))
        print("Merge:", benchmark_sort(merge_sort, mileage))

def main(): # main function to run the whole program
    collection = load_data("vehicles.csv") # loads the dataset
    experiment_by_price(collection) # runs the experiments
    experiment_by_year(collection)

if __name__ == "__main__":
    main()
```

References

Brightspace. (2026). Lecture notes and lab tutorial materials.
Available via: TU Dublin Brightspace
Accessed: 21 March 2026

GitHub link: https://github.com/bytelea/Sorting-Algorithms-Analysis-in-Python

DataCamp. (2026). Data Structures and Algorithms in Python – Statement of Accomplishment. Available at: https://www.datacamp.com/statement-of-accomplishment/course/98d0b7777be02ab368e3c86c4ab8570d8481bdbd
Accessed: 19 March 2026

GeeksforGeeks. Sorting Algorithms.
Available at: https://www.geeksforgeeks.org/dsa/sorting-algorithms/
Accessed: 21 March 2026

GeeksforGeeks. Bubble Sort Algorithm.
Available at: https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/
Accessed: 21 March 2026

GeeksforGeeks. Selection Sort Algorithm.
Available at: https://www.geeksforgeeks.org/dsa/selection-sort-algorithm-2/
Accessed: 21 March 2026

GeeksforGeeks. Insertion Sort Algorithm.
Available at: https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/
Accessed: 21 March 2026

GeeksforGeeks. Quick Sort Algorithm.
Available at: https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/
Accessed: 21 March 2026

GeeksforGeeks. Merge Sort Algorithm.
Available at: https://www.geeksforgeeks.org/dsa/merge-sort/
Accessed: 21 March 2026

GeeksforGeeks. Sorting Algorithms in Python.
Available at: https://www.geeksforgeeks.org/python/sorting-algorithms-in-python/ Accessed: 22 March 2026

NeetCode. Sorting Algorithms Course.Available at: https://neetcode.io/courses/lessons/sorting-algorithms
Accessed: 22 March 2026

Algocademy. Sorting Algorithms Explained from Bubble Sort to Quick Sort. Available at: https://algocademy.com/blog/sorting-algorithms-explained-from-bubble-sort-to-quick-sort/
Accessed: 22 March 2026

Real Python. Sorting Algorithms in Python.
Available at: https://realpython.com/sorting-algorithms-python/
Accessed: 22 March 2026

Furkan Torcuk. An Academic Perspective on Sorting Algorithms.
Available at: https://medium.com/@furkantorcuk1/an-academic-perspective-on-sorting-algorithms-foundations-comparisons-and-implementations-0e597eefd3a0 Accessed: 22 March 2026

GitHub. 14 Patterns Coding Interview Question.
Available at: https://github.com/chamanlal97196/14-Patterns-Coding-Interview-Question
Accessed: 21 March 2026